

# Aeternum-core Smart Contract

## Technical Documentation

**Version:** 1.0

**Network:** Ethereum

**Contract:** AeternumVault.sol

**Prepared by:** Ndubuisi Ugwuja — Founder, Aeternum Labs

---

## Overview

AeternumVault is a smart wallet vault built on Ethereum. It lets you store ETH safely, send and receive funds like a normal wallet, and set up an automatic backup plan in case you ever lose access to your wallet — or in case something happens to you.

Here is the core idea in plain terms:

You deposit ETH into your vault. You pick a backup address — a wallet owned by someone you trust, like a family member, or a second wallet you control. You set a timer, for example 365 days. If you never interact with your vault for that entire period, Chainlink Automation (a decentralized service that watches the blockchain) automatically sends your ETH to your backup address.

If you are still alive and active, you just ping the contract occasionally to reset the timer. No one can touch your funds except you. There are no admins, no custodians, and no backdoors.

---

## How It Works

### The Vault

When you register with AeternumVault, the contract creates a private vault in your name. Think of it like a safety deposit box with your name on it. Your ETH sits inside, growing in value with the broader market, and only you can access it under normal conditions.

You can use your vault like a normal wallet:

- **Deposit** ETH into it at any time
- **Send** ETH to any address directly from your vault
- **Withdraw** all your ETH back to your own wallet in one click
- **Update** your backup address or inactivity timer whenever you want

- **Cancel** your setup entirely and get all your ETH back

## The Inactivity Timer

Every time you interact with your vault — deposit, send, withdraw, or even just call the ping function — the timer resets. The timer counts from your last interaction.

If your timer reaches zero (meaning you have been completely inactive for your chosen period), Chainlink Automation will detect this and send your ETH to your backup address automatically.

If you interact with your vault even once before the timer runs out, the timer resets and the process starts over.

## The Ping Function

The ping function exists for one purpose: to prove you are still alive and active without moving any funds. Calling ping costs very little gas (around 6,000 gas units — a fraction of a cent) and resets your inactivity timer. If you go on a long trip, have a medical situation, or simply do not need to move funds for a while, just ping your vault to keep the timer fresh.

## Subscription Tiers

AeternumVault has two tiers:

### Free Tier

- Minimum inactivity period of 365 days (1 year)
- No monthly fees
- All core features included

### Premium Tier

- Minimum inactivity period of 180 days (6 months)
- Costs 0.002 ETH per month
- Allows a shorter timer — useful if you want faster recovery in case of emergency

The only practical difference between the two tiers is the minimum timer length. Premium users can set a shorter inactivity period.

---

## Key Actors

### You (The User)

You control your vault entirely. Only you can deposit, send, withdraw, update your settings, or cancel your vault. No one else — not even the contract owner — can access your funds.

## Your Backup Address

This is the wallet that receives your ETH if you go inactive for too long. It can be:

- A wallet held by a trusted family member
- A hardware wallet you keep in cold storage
- A multisig wallet shared with people you trust
- Any Ethereum address that can receive ETH

Your backup address cannot access your vault while you are active. It only receives funds if the inactivity timer expires and the automatic recovery runs.

## Chainlink Automation

Chainlink Automation is a decentralized network of nodes that watches the blockchain for conditions you define. In AeternumVault, it watches all registered vaults and checks whether any have been inactive long enough for recovery to trigger.

Chainlink nodes call a function called `checkUpkeep` every 12 seconds. If any wallets are due for recovery, they call `performUpkeep` to execute the ETH transfer to the backup address.

This is fully automated. No human being decides when to send your ETH — the code does it automatically based on the rules you set.

## The Treasury

AeternumVault collects subscription fees from Premium tier users. These fees go into a separate treasury account that is completely isolated from user vault balances. The treasury address can only withdraw subscription revenue — it cannot touch any user's deposited ETH under any circumstances.

---

# Automatic Recovery

## How Recovery Is Triggered

Recovery happens automatically when all three of the following are true at the same time:

1. Your vault is registered and active
2. Your vault has a non-zero ETH balance
3. Your inactivity period has fully elapsed since your last interaction

When Chainlink detects this, it sends your entire ETH balance to your backup address in a single transaction.

## What Happens If the Transfer Fails

Sometimes a backup address cannot receive ETH — for example, if it is a contract that was set up incorrectly. In this case, the recovery attempt fails and is counted as one failed attempt.

AeternumVault allows up to 3 failed recovery attempts. After the third consecutive failure, your vault is permanently marked as abandoned. This means:

- Your vault is removed from the automatic monitoring system (so Chainlink stops retrying)
- Your ETH balance is still fully accessible to you — you can withdraw it or send it at any time
- You cannot re-register until you fix the issue with your backup address

This 3-attempt limit exists to prevent Chainlink from running in an infinite loop trying to send ETH to a broken address, which would waste LINK (the token Chainlink uses to pay for operations).

---

## Architecture

### Single Contract

AeternumVault is a single smart contract. There are no complex protocols, no liquidity pools, no lending, and no borrowing. Your ETH sits in the contract mapped to your address. It is never mixed with anyone else's funds.

Each vault is completely isolated. If something goes wrong with one user's setup, it has zero effect on any other user's vault.

### The Registry

The contract keeps a list of all registered wallets that are currently active. This list is how Chainlink knows which wallets to monitor.

The registry uses an efficient data structure that allows wallets to be added and removed in  $O(1)$  time — meaning it takes the same amount of computing resources regardless of how many wallets are registered. This keeps gas costs manageable even when the protocol has hundreds of thousands of users.

### Pagination for Scale

The `checkUpkeep` and `performUpkeep` functions both support pagination. Instead of trying to check all wallets at once (which would exceed Ethereum's gas limits), Chainlink can check and process wallets in batches of up to 50 at a time. For a protocol with 500,000 users, multiple Chainlink upkeep jobs can run in parallel, each covering a different range of wallets.

---

# Security Design

## You Own Your Funds

AeternumVault is non-custodial. This means the protocol never owns your ETH — you do. The contract simply holds it on your behalf and executes your instructions. There is no admin key that can move your funds. There is no pause button that can freeze your vault. There is no upgrade mechanism that can change the rules after you have deposited.

What you see in the contract is what you get — permanently.

## Reentrancy Protection

A reentrancy attack is a type of hack where a malicious contract tries to call back into a function before the first call has finished, allowing an attacker to drain funds. AeternumVault uses two independent defences against this:

1. **Checks-Effects-Interactions (CEI) pattern:** The contract always updates all balances and state before sending any ETH. By the time any ETH transfer happens, your balance is already recorded as zero — so even if a malicious backup address tried to re-enter the contract, there is nothing left to steal.
2. **ReentrancyGuard:** Every function that sends ETH is protected by OpenZeppelin's ReentrancyGuard, a battle-tested lock that blocks re-entry at the code level.

## Access Control

The contract uses two access control modifiers:

- **onlyRegistered:** Only wallets that have registered a vault can call deposit, send, withdraw, ping, and configuration functions. An unregistered address calling these functions will be rejected.
- **onlyTreasury:** Only the treasury address can withdraw subscription fees. The treasury address cannot access any user's vault balance.

There are no other privileged roles. No owner. No admin. No pause function.

## Direct ETH Rejection

The contract's `receive()` function is set to revert. This means if you accidentally send ETH directly to the contract address (without going through the deposit function), the transaction fails immediately and your ETH is returned. This prevents accidental fund lock.

---

## Constants and Limits

| Setting                 | Value           | What It Means                                                         |
|-------------------------|-----------------|-----------------------------------------------------------------------|
| Minimum timer (Free)    | 365 days        | Free tier users must wait at least 1 year before recovery can trigger |
| Minimum timer (Premium) | 180 days        | Premium users can set a timer as short as 6 months                    |
| Maximum timer           | 3,650 days      | No vault can have a timer longer than 10 years                        |
| Subscription duration   | 30 days         | Premium subscription renews monthly                                   |
| Premium fee             | 0.002 ETH/month | Cost of the Premium tier                                              |
| Maximum batch size      | 50 wallets      | Chainlink processes at most 50 wallets per execution call             |
| Maximum failed attempts | 3               | After 3 failed recovery transfers, the vault is abandoned             |

---

## Chainlink Automation Integration

### How Chainlink Watches Your Vault

Chainlink Automation operates in two steps:

**Step 1 — `checkUpkeep` (off-chain, free)** Chainlink nodes simulate a call to `checkUpkeep` every 12 seconds. This function scans registered wallets and checks whether any are due for recovery. This simulation costs nothing — it runs off-chain and does not spend any gas.

**Step 2 — `performUpkeep` (on-chain, costs LINK)** If `checkUpkeep` finds wallets due for recovery, Chainlink submits a `performUpkeep` transaction on-chain. This is the real transaction that actually sends ETH to backup addresses. LINK is deducted from the upkeep balance to pay for this.

### Stale Data Safety

There is a gap between when `checkUpkeep` identifies a wallet and when `performUpkeep` actually executes. In this gap, a user might have already pinged their vault or made a withdrawal, making

recovery no longer needed. AeternumVault handles this safely: performUpkeep re-validates every wallet individually before sending any ETH. If a wallet is no longer due for recovery, it is silently skipped and no ETH is sent.

## Setting Up Chainlink Automation

After deploying the contract:

1. Go to [automation.chain.link](https://automation.chain.link) and connect your wallet
2. Click Register new upkeep and select Custom Logic
3. Enter your contract address
4. Set checkData to `abi.encode(uint256(0), uint256(50))` to start at wallet 0 with a batch size of 50
5. Set the gas limit to 5,000,000
6. Fund the upkeep with Sepolia LINK from [faucets.chain.link](https://faucets.chain.link)

For large user counts, register multiple upkeep jobs covering different ranges of wallets. For example, one job covering wallets 0–49, another covering wallets 50–99, and so on.

---

## Events

The contract emits the following events so that frontends, indexers, and monitoring tools can track everything that happens:

| Event                     | When It Is Emitted                                             |
|---------------------------|----------------------------------------------------------------|
| RecoveryRegistered        | A user successfully registers a vault                          |
| Deposited                 | ETH is deposited into a vault                                  |
| Sent                      | ETH is sent from a vault to another address                    |
| Withdrawn                 | A user withdraws all ETH from their vault                      |
| ActivityPinged            | A user calls ping to reset their timer                         |
| BackupAddressUpdated      | A user changes their backup address                            |
| InactivityPeriodUpdated   | A user changes their timer length                              |
| SubscriptionRenewed       | A Premium user renews their subscription                       |
| RecoveryExecuted          | Chainlink successfully sends ETH to a backup address           |
| RecoveryFailed            | A recovery attempt fails (backup address cannot receive ETH)   |
| RecoveryAbandoned         | A vault reaches 3 failed attempts and is permanently abandoned |
| RecoveryCancelled         | A user cancels their vault and withdraws their ETH             |
| SubscriptionFeesWithdrawn | The treasury withdraws accumulated subscription fees           |
| TreasuryUpdated           | The treasury address is updated                                |

---

# User Functions

## **register(backupAddress, inactivityPeriod, tier)**

Creates your vault. You provide your backup address, how long the inactivity timer should be (in seconds), and whether you want Free or Premium tier. You can optionally deposit ETH at the same time by sending it with the transaction.

## **deposit()**

Adds ETH to your existing vault. Send ETH with this transaction and it will be added to your vault balance. Your inactivity timer resets.

## **send(to, amount)**

Sends ETH from your vault to any Ethereum address. Your vault balance decreases by the amount sent. Your inactivity timer resets.

## **withdrawAll()**

Sends your entire vault balance back to your own wallet and your inactivity timer resets.

## **ping()**

Resets your inactivity timer without moving any funds. Costs minimal gas. Use this to prove you are active without making any transfers.

## **updateBackupAddress(newBackupAddress)**

Changes your backup address to a new one. Your timer resets.

## **updateInactivityPeriod(newPeriod)**

Changes how long your inactivity timer is. Must be within the allowed range for your tier. Your timer resets.

## **renewSubscription()**

Extends your Premium subscription by 30 days. Send 0.002 ETH with this transaction.

## **cancelRecovery()**

Withdraws all your ETH and removes your vault from the monitoring system in a single transaction. This is the clean exit function.

---

## Read-Only Functions

### **isRegistered(wallet)**

Returns true if the given wallet has an active vault registered.

### **getRecoveryConfig(wallet)**

Returns the full configuration of a vault: backup address, inactivity period, current balance, last activity timestamp, tier, subscription expiry, active status, failed attempt count, and abandoned status.

### **isRecoveryDue(wallet)**

Returns true if a vault is currently due for recovery (inactivity period has elapsed, balance is non-zero, vault is active).

### **getTimeUntilRecovery(wallet)**

Returns how many seconds remain before this vault becomes eligible for recovery. Returns 0 if recovery is already due.

### **getTotalRegistered()**

Returns how many wallets are currently registered and active in the protocol.

### **getRegisteredWallets(start, end)**

Returns a slice of the registered wallets array. Used for pagination.

### **getAccumulatedFees()**

Returns the total subscription fees collected and not yet withdrawn.

### **getTreasury()**

Returns the current treasury address.

---

# Known Limitations

**ETH only:** The current version of AeternumVault supports native ETH only. ERC-20 token support (USDC, WBTC, etc.) is planned for a future version.

**No yield:** ETH sitting in your vault does not earn interest in the current version. Future versions plan to integrate with Aave and Lido to generate yield on deposited ETH while it waits.

**EOA wallets only:** You must interact with the contract using a standard Ethereum wallet (MetaMask, hardware wallet, etc.). ERC-4337 smart wallet abstraction is not yet supported.

**External audit pending:** AeternumVault has undergone comprehensive internal security testing including 109 unit tests, invariant testing, static analysis with Slither, and property-based fuzzing with Echidna across 50,000+ randomized call sequences. An independent external audit by a professional security firm is required before mainnet deployment.

---

## Next Phase

### Aeternum v2:

Expanding the protocol from a passive recovery vault into a **yield-generating recovery system** that supports ERC20 tokens, enabling deposited assets to earn yield via Aave and Lido integration while maintaining automated inheritance guarantees, along with MetaMask Snap integration for a tighter wallet experience. Users can optionally choose to earn yield on their deposits or stick to the V1 philosophy of retrieving their native ETH and ERC-20 assets if recovery is triggered.

#### Core upgrades include:

- ERC-20 asset support
- Yield generation via Aave and Lido integrations
- MetaMask Snap integration for improved wallet UX

#### Aave Integration (Lending Yield):

ETH deposits are optionally routed into Aave to generate variable lending yield through overcollateralized liquidity markets.

#### Flow:

1. User deposits ETH into Aeternum v2
2. Contract wraps ETH → WETH
3. WETH is supplied to Aave Pool

4. Contract receives aWETH (interest-bearing ERC-20 token), which is held in the user's vault

**Recovery Execution:**

1. On inactivity trigger, contract calls Aave Pool withdraw()
2. Redeems WETH (principal + accrued yield)
3. Unwraps WETH → ETH
4. Transfers ETH to backup address

**Key Considerations:**

- Withdrawal depends on Aave pool liquidity
- Yield is variable and not guaranteed

## Lido Integration (Staking Yield)

ETH deposits are optionally staked through Lido to earn Ethereum staking rewards in a liquid staking model.

**Flow:**

1. User deposits ETH into Aeternum v2
2. Contract stakes ETH via Lido
3. Contract receives stETH (rebasing staking token), which is held in the user's vault

**Recovery Execution:**

1. Upon inactivity trigger, contract initiates stETH unwinding
2. Converts stETH (principal + staking rewards) → ETH
3. ETH is transferred to the designated backup address

**Key Considerations:**

- Conversion from stETH depends on Lido redemption mechanics and/or market liquidity conditions
- stETH introduces peg and liquidity risk until fully converted

---

## Frequently Asked Questions

**Can the protocol take my ETH?** No. There is no admin key, no owner function, and no mechanism of any kind that allows anyone — including the people who built this — to access your vault balance. Your ETH can only go to you or your designated backup address.

**What happens if I forget to ping and my timer expires?** Your ETH is sent to your backup address automatically. If your backup address is a wallet you also control, you can simply move the ETH back. If you want to continue using the vault, register again with a new vault.

**What if I lose access to my backup address too?** This is why choosing your backup address carefully matters. We recommend using a hardware wallet in cold storage, a multisig wallet with multiple signers, or a wallet held by a trusted person. If both your primary and backup wallets are lost, the ETH cannot be recovered.

**What if Chainlink goes down?** Chainlink is a decentralized network with thousands of nodes. A complete outage is extremely unlikely. However, if Chainlink were somehow unavailable, your ETH would simply stay in your vault until service resumes. The worst case is a delay in recovery — your funds are never at risk.

**Can I have multiple vaults?** Currently, each Ethereum address can register one vault. Multiple vaults per address are planned for a future version.

**Is the contract upgradeable?** No. AeternumVault is immutable. Once deployed, the code cannot be changed. This is intentional — it means the rules you signed up for are the rules that apply forever, and no governance vote or team decision can change how the contract behaves.

---

*AeternumVault is built by Aeternum Labs. © 2026 Aeternum Labs. All rights reserved.*